PROGRAMMING IN JAVA

ASSIGNMENT 2 – CO2

Complete Answer Guide (Q1–Q7)

Name: Yuvaraj D

Register Number: 192421230

Course: CSA09 – Programming in Java

Topics: Strings, Collections, Maps, Generics, Comparators and Priority Queues

Important submission note: This report provides explanations, designs, complexity analysis and runnable Java examples. The assignment specifically requires your own repeated timing measurements, output screenshots and GitHub commits. Do not present invented timings as measured results. Run the benchmark programs on your computer and enter your actual results in the evidence tables.

Q1 – Building Text Under Load: String, StringBuilder or StringBuffer?

1. Prediction and explanation

String is immutable. Every concatenation with + creates a new String result, so repeatedly appending fields copies the text built so far. For n similarly sized fields, the total copied characters can grow quadratically. StringBuilder is mutable and appends into a growable character buffer; it is normally the fastest choice for a local, single-threaded ticket. StringBuffer is also mutable, but its methods are synchronized, so it adds synchronization overhead and is useful when the same buffer is genuinely shared between threads.

2. Three workloads

Workload	Test size	What it reveals
A – small	1 ticket × 6 fields	Differences may be negligible; setup overhead can dominate.
B – heavy	10,000 tickets × 50 fields	Repeated copying and allocation become visible.
C – custom	1,000 tickets × 200 fields	Tests many appends per ticket with larger output.

3. Example implementation

```
import java.util.*;

public class TicketBuild {
    static String[] fields(int count) {
        String[] a = new String[count];
        for (int i = 0; i < count; i++) a[i] = "FIELD" + i + "|";
        return a;
    }
    static String usingPlus(String[] a) {
        String s = "";
        for (String x : a) s += x;
        return s;
    }
    static String usingBuilder(String[] a) {
        StringBuilder b = new StringBuilder();
        for (String x : a) b.append(x);
        return b.toString();
    }
    static String usingBuffer(String[] a) {
        StringBuffer b = new StringBuffer();
        for (String x : a) b.append(x);
        return b.toString();
    }
    public static void main(String[] args) {
        String[] a = fields(50);
        int tickets = 10000;
        long t1 = System.nanoTime();
        for (int i=0;i<tickets;i++) usingPlus(a);
        long t2 = System.nanoTime();
        for (int i=0;i<tickets;i++) usingBuilder(a);
        long t3 = System.nanoTime();
        for (int i=0;i<tickets;i++) usingBuffer(a);
        long t4 = System.nanoTime();
        System.out.println("String +: " + (t2-t1)/1_000_000.0 + " ms");
        System.out.println("StringBuilder: " + (t3-t2)/1_000_000.0 + " ms");
        System.out.println("StringBuffer: " + (t4-t3)/1_000_000.0 + " ms");
    }
}
```

For a fair experiment, warm up the JVM, repeat each measurement at least 5–10 times, rotate the order of tests, and compare averages or medians. Keep the same fields and ticket count for all three methods. Use System.nanoTime(); memory readings from Runtime are approximate and affected by garbage collection.

4. Design decision

• Use StringBuilder for each ticket because each ticket is built locally by one thread.

• Use a dedicated synchronized logger, or a thread-safe logging framework, for a shared booking log. If a shared StringBuffer is specifically required, its synchronized append operations protect individual calls, but a multi-step record

operation may still need external synchronization to prevent interleaving.

5. Complexity and alternatives

Repeated + in a loop can copy 1 + 2 + … + n portions of text, giving O(n²) character-copy work for similarly sized fields. StringBuilder appends are amortized O(1) per append and O(n) overall in total output length. StringJoiner or String.join() is convenient when joining a known collection of fields with a delimiter; String.format() is readable for formatted output but generally has more formatting overhead. StringBuilder uses a resizable backing storage plus the final String; repeated + creates many intermediate String objects.

Evidence table to complete after running:

Workload	String + avg ms	StringBuilder avg ms	StringBuffer avg ms	Memory / notes
A – 1 × 6	Your result	Your result	Your result	Your result
B – 10,000 × 50	Your result	Your result	Your result	Your result
C – 1,000 × 200	Your result	Your result	Your result	Your result

Q2 – When Is ArrayList an Appropriate Engineering Choice?

1. Comparison

Operation	ArrayList	LinkedList	ArrayDeque
Add at end	O(1) amortized	O(1)	O(1) amortized at either end
Add at front	O(n) shifting	O(1)	O(1) amortized
Insert in middle	O(n) shifting	O(n) to reach position; O(1) link once there	Not designed for indexed middle insertion
Remove at front	O(n) shifting	O(1)	O(1)
Get k-th	O(1)	O(n)	No indexed get; O(n) traversal
Full iteration	O(n), cache-friendly	O(n), node traversal	O(n)

ArrayList is a good choice when the application frequently reads by index, iterates, and appends, while insertions/removals near the beginning are uncommon. Stop recommending it when front or middle modifications dominate and shifting a large suffix becomes a measurable bottleneck. LinkedList has extra node objects and pointer memory; ArrayList stores references in a contiguous resizable array.

2. Experiment design

Test sizes: 1,000; 10,000; and 100,000 songs. For each size, separately measure end-add, front insert/remove, middle insert/remove, random get(k), and full iteration. Prebuild equivalent lists, use the same random indices, warm up, repeat trials and report average/median. Avoid timing list construction together with the operation being tested.

3. Changed condition: Up Next

For frequent additions at both ends and removals from the front, ArrayDeque is usually the best fit. It supports efficient addFirst, addLast, removeFirst and removeLast. It does not provide indexed access or arbitrary middle insertion, so retain ArrayList for the editable playlist if random indexed access remains important.

4. Plain array alternative

A fixed plain array requires manual shifting: inserting at index k shifts n–k elements right; removing at k shifts n–k–1 elements left. Both are O(n). Reading index k is O(1), and iteration is O(n). A dynamically resized array behaves like ArrayList, including amortized O(1) append and occasional O(n) resize.

Engineering conclusion: Use ArrayList for the main playlist and ArrayDeque for the Up Next queue if both usage patterns exist.

Q3 – Engineer the hashCode–equals Contract

1. Why the original class is unsafe

equals() considers two Student objects equal when their registration numbers match. HashSet first uses hashCode() to select a bucket and then uses equals() among candidates in that bucket. If equal objects produce different hash codes, they may be placed in different buckets and both remain in the set. Therefore equal objects must always have equal hash codes.

2. Three test designs and predictions

• Test A – same registration number and same name, with equal hash codes: expected set size 1.

• Test B – same registration number but different names: because identity is defined by regNo, expected set size 1 after the contract is fixed.

• Test C – mutate regNo after insertion: changing a field used by equals/hashCode can make the object difficult to find or remove from its original bucket. Prefer immutable identity fields.

3. Corrected complete program

```
import java.util.HashSet;
import java.util.Set;
import java.util.Objects;

class Student {
    private final String regNo;
    private final String name;

    Student(String regNo, String name) {
        this.regNo = regNo;
        this.name = name;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;
        if (!(o instanceof Student)) return false;
        Student other = (Student) o;
        return Objects.equals(regNo, other.regNo);
    }

    @Override
    public int hashCode() {
        return Objects.hash(regNo);
    }

    @Override
    public String toString() {
        return regNo + " " + name;
    }
}

public class Attendance {
    public static void main(String[] args) {
        Set<Student> present = new HashSet<>();
        present.add(new Student("192311001", "Arun"));
        present.add(new Student("192311002", "Divya"));
        present.add(new Student("192311001", "Arun"));
        System.out.println("Unique students: " + present.size());
    }
}
```

Expected output: Unique students: 2

Before hashCode() is implemented, the exact output is not guaranteed: Object's default hash code is generally identity-based, so the duplicate may be stored. With the corrected method, equal registration numbers have equal hash codes and the expected size is 2.

4. Alternative structures and complexity

TreeSet with a Comparator on regNo stores one entry per comparison-equivalent registration number and costs O(log n) per add/search. HashMap keyed by regNo provides O(1) average put/get and makes the uniqueness key explicit. Both use O(n) storage. HashSet also provides O(1) average add/contains, assuming a good hash distribution. Never mutate fields used by equals/hashCode while an object is stored in a hash collection.

Q4 – Choosing a Map for a Pharmacy Inventory

1. Map comparison

Map	put/get	Iteration order	Best use
HashMap	O(1) average	No guaranteed order	Fast stock lookup
TreeMap	O(log n)	Sorted by key	Code-order report
LinkedHashMap	O(1) average	Insertion order by default	Added-order report

A TreeMap naturally produces the medicine-code report. A LinkedHashMap naturally produces the order-added report. HashMap is suitable for fast lookup but needs a separate sort of keys (O(n log n)) for a code-order report and cannot guarantee insertion order. If both reports and fast lookup are essential, maintain a HashMap for lookup and an ordered structure for reporting, or choose a structure based on the dominant requirement.

2. Java example

```
import java.util.*;

public class PharmacyMaps {
    public static void main(String[] args) {
        Map<String, Integer> hash = new HashMap<>();
        Map<String, Integer> sorted = new TreeMap<>();
        Map<String, Integer> added = new LinkedHashMap<>();

        String[] codes = {"M103", "M101", "M102"};
        int[] stock = {25, 40, 18};

        for (int i=0; i<codes.length; i++) {
            hash.put(codes[i], stock[i]);
            sorted.put(codes[i], stock[i]);
            added.put(codes[i], stock[i]);
        }

        System.out.println("Lookup M102: " + hash.get("M102"));
        System.out.println("Code order: " + sorted);
        System.out.println("Added order: " + added);
    }
}
```

Expected output: Lookup M102: 18; Code order: {M101=40, M102=18, M103=25}; Added order: {M103=25, M101=40, M102=18}.

3. Measurement plan and deployment

Test 1,000; 10,000; and 100,000 medicines. Measure 100,000 lookups using the same pre-generated key sequence; separately time the code-order and insertion-order reports. Define an acceptable lookup target before testing (for example, under 5 ms for a batch of 100,000 lookups on the test machine; state the hardware and JVM). Use repeated runs and actual local results. Hashtable is generally not preferred in new code because it is a legacy synchronized collection; modern choices offer clearer concurrency options and better collection APIs.

4. Sorted array alternative

Keep an array sorted by medicine code and use Arrays.binarySearch(): lookup O(log n), insertion O(n) due to shifting, code-order report O(n) because the array is already sorted, and storage O(n). HashMap gives O(1) average lookup but no sorted order; TreeMap gives O(log n) lookup and maintains sorted order at O(n) storage.

Q5 – Derive a Generic Ranking Component from a Principle

1. Design

Use a separate Comparator because the same Team data may be ranked in different ways, and the component should also rank players, colleges and quiz teams. Leaderboard accepts Comparator, so a comparator for T or one of its supertypes can be used. List is useful when copying items from a list of T or a subtype into a destination that stores T.

2. Complete runnable program

```
import java.util.*;
```

```
class Team {
    final String name;
    final int points;
    final int goalDiff;

    Team(String name, int points, int goalDiff) {
        this.name = name;
        this.points = points;
        this.goalDiff = goalDiff;
    }

    public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;

    Leaderboard(Comparator<? super T> order) {
        this.order = order;
    }

    void add(T item) {
        items.add(item);
    }

    List<T> ranking() {
        List<T> copy = new ArrayList<>(items);
        copy.sort(order);
        return copy;
    }
}

public class LeaderboardDemo {
    public static void main(String[] args) {
        Comparator<Team> byRank =
            Comparator.comparingInt((Team t) -> t.points).reversed()
                .thenComparing(
                    Comparator.comparingInt((Team t) -> t.goalDiff).reversed())
                .thenComparing(t -> t.name);

        Leaderboard<Team> board = new Leaderboard<>(byRank);
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));
        System.out.println(board.ranking());
    }
}
```

Expected output: [Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]

Bisons ranks above Panthers because both have 9 points and goal difference 4, so the final alphabetical tie-breaker places Bisons before Panthers.

3. Correctness and complexity

The comparator applies a lexicographic order: compare points descending; only if tied compare goal difference descending; only if still tied compare name ascending. Lexicographic composition of consistent field comparisons is transitive and deterministic for distinct names. Avoid subtraction such as b.points - a.points because integer overflow can produce incorrect results; comparing methods avoid that risk.

Sorting a copy costs O(n log n) time and O(n) extra space, leaving the stored insertion list unchanged. Comparable is suitable when one natural order is universally meaningful. Separate Comparator objects are easier to reuse for players/colleges or alternative ranking policies. Three stable sorts also cost about 3 × O(n log n) and are less direct; stable sorts must be applied from least-significant key to most-significant key.

Q6 – Priority, Order and Fairness in a Queue

1. Design

A normal FIFO Queue cannot choose the most severe patient first because arrival order alone controls removal. PriorityQueue orders by a comparator. Use severity ascending (1 is most urgent), then arrival counter ascending to preserve FIFO order among equal severities.

2. Complete runnable program

```
import java.util.Comparator;
import java.util.PriorityQueue;

class Patient {
    final String name;
    final int severity;
    final long arrival;

    Patient(String name, int severity, long arrival) {
        this.name = name;
        this.severity = severity;
        this.arrival = arrival;
    }

    public String toString() {
        return name + "(S" + severity + ")";
    }
}

public class Triage {
    private long counter = 0;
    private final PriorityQueue<Patient> queue =
        new PriorityQueue<>(
            Comparator.comparingInt((Patient p) -> p.severity)
                      .thenComparingLong(p -> p.arrival));

    void admit(String name, int severity) {
        queue.add(new Patient(name, severity, counter++));
    }

    Patient next() {
        return queue.poll();
    }

    public static void main(String[] args) {
        Triage er = new Triage();
        er.admit("Ravi", 3);
        er.admit("Meena", 1);
        er.admit("John", 3);
        er.admit("Asha", 5);
        er.admit("Kumar", 1);
        er.admit("Lata", 2);

        StringBuilder order = new StringBuilder();
        Patient p;
        while ((p = er.next()) != null) order.append(p).append(' ');
        System.out.println(order.toString().trim());
    }
}
```

Expected output: Meena(S1) Kumar(S1) Lata(S2) Ravi(S3) John(S3) Asha(S5)

The counter increases for every admission. If two patients have equal severity, the smaller counter belongs to the earlier arrival and is removed first. Printing PriorityQueue directly does not guarantee sorted treatment order; repeatedly poll() to obtain priority order.

3. Complexity, starvation and scale

PriorityQueue insertion and poll are O(log n); peek is O(1); storage is O(n). A LinkedList maintained in sorted order has O(n) insertion and O(1) removal from the front. Five FIFO queues, one per severity, allow O(1) enqueue and dequeue; checking the five queue heads is O(5), effectively O(1), with O(n) total storage.

Starvation can occur because a steady stream of critical cases may continually move ahead of minor cases. A fair policy can increase a patient's effective priority after a waiting-time threshold, while preserving clinical safety. For example, periodically promote patients who have waited longer than 30 minutes, subject to doctor review and triage rules.

A city hospital receiving 5,000 patients per day averages about 0.058 patients per second over 24 hours; even peak periods are manageable for an O(log n) priority queue in ordinary application terms. Real deployment must also consider

concurrent admissions, persistence, audit logs, emergency overrides, reliability and clinical governance.

Q7 – Diagnose, Redesign and Validate

Chosen case: playlist editing with frequent front removals

This example selects Q2's ArrayList-based playlist under the changed workload. In the original design, removing the first song shifts every remaining reference one position to the left. For n songs, each front removal performs O(n) shifting; repeatedly removing many front items can create $O(n^2)$ total work.

A. Diagnose the bottleneck

• Most extra work is reference shifting after remove(0).

• The operation that scales poorly is front removal; its cost grows linearly with playlist size.

• The limitation comes from ArrayList's contiguous indexed storage, which preserves index positions by shifting elements.

B. Design objective

For an Up Next queue with frequent additions at both ends and removals from the front, target amortized O(1) insertion/removal at either end, while preserving FIFO order. Validate using the same sequence of operations at 1,000, 10,000 and 100,000 elements.

C. Revised solution and pseudocode

```
Use ArrayDeque<Song> upNext;

add to front:  upNext.addFirst(song)
add to back:   upNext.addLast(song)
play next:     song = upNext.pollFirst()
inspect next:  song = upNext.peekFirst()
```

ArrayDeque uses a resizable circular array, so it avoids shifting the whole collection for ordinary front removals. It uses O(n) total storage. It does not support indexed get(k), null elements or arbitrary middle insertion; those remain reasons to keep ArrayList for the main playlist.

D. Prediction

For repeated remove-first operations, ArrayDeque should take approximately linear total time across m removals (amortized O(1) each), while ArrayList can require O(mn) shifting when removing m elements from a list initially sized n.

E. Implementation and validation

```
import java.util.*;

public class QueueRedesign {
    public static void main(String[] args) {
        int n = 100000;

        ArrayList<Integer> list = new ArrayList<>();
        ArrayDeque<Integer> deque = new ArrayDeque<>();

        for (int i = 0; i < n; i++) {
            list.add(i);
            deque.addLast(i);
        }

        long a = System.nanoTime();
        while (!list.isEmpty()) list.remove(0);
        long b = System.nanoTime();

        long c = System.nanoTime();
        while (!deque.isEmpty()) deque.removeFirst();
        long d = System.nanoTime();

        System.out.println("ArrayList front removals: " + (b-a)/1_000_000.0 + " ms");
        System.out.println("ArrayDeque front removals: " + (d-c)/1_000_000.0 + " ms");
    }
}
```

Run each test multiple times after warm-up. Use identical initial data and operation counts. Do not use a single run as proof; record averages/medians and machine/JDK details.

F. Final evaluation

If measurements show lower time for ArrayDeque, the prediction is supported. The trade-off is loss of indexed access and middle insertion. ArrayList remains preferable for a playlist that mostly reads by index, iterates and appends. The correct engineering choice is therefore not to replace every list: use ArrayList for the editable playlist and ArrayDeque for the Up Next queue.

Final submission checklist

• Create repository: CSA09-192421230-Assignment2.

• Create separate folders Task01 through Task07 (or Q1 through Q7).

• Add Java source files, input data, output screenshots and raw benchmark results.

• Add README.md with name, register number, Java version, compile/run steps and result notes.

• Commit progressively, with at least one meaningful commit per task.

• Replace every 'Your result' field with measurements from your own computer; include repeated runs and explain unusual results.

• Add the working GitHub URL to the report cover page and assignment submission.

Compile/run reminder: Save each public class in a file with the same name (for example, Attendance.java for public class Attendance). Compile with `javac Attendance.java` and run with `java Attendance`. Use a separate folder for each program to avoid class-name conflicts.

End of answer guide.